

You already know that a program can create an object at any time. Often, a program will create one or more objects at startup and use those objects throughout its operation. However, in many cases a program will want to create an object when it needs it, but release or remove the object when it's done using it to free up the resources that the object uses (memory, files, network connections, and so on). Here are a few examples:

- A “transaction” object that’s used while a customer is making an electronic purchase. When the purchase is completed, the object is destroyed.
- An object to handle communication over the internet that’s released when the communication is completed.
- Transient objects in a game. The program could instantiate many copies of bad guys, aliens, spaceships, and so on; as the player destroys each one, the program can eliminate the underlying object.

The period of time from the instantiation of an object until it is destroyed is known as the object’s *lifetime*. To understand the lifetime of an object, you first need to know about a related underlying concept having to do with the implementation of objects in Python (and some other OOP languages): the reference count.

Reference Count

There are a number of different implementations of Python. The following discussion of reference counts applies to the official version released by the Python Software Foundation—the version downloaded from python.org—that’s commonly known as *CPython*. Other implementations of Python may use a different approach.

Part of the philosophy of Python is that programmers should never have to worry about the details of managing memory. Python takes care of that for you. However, having a basic idea of how Python manages memory will be helpful in understanding how and when objects are released back to the system.

Whenever a program instantiates an object from a class, Python allocates memory for the storage of the instance variables defined in the class. Each